Khulna University of Engineering & Technology

Department of Computer Science and Engineering

Course Code: CSE-4102

Course Title: Computer Graphics and Image Processing Laboratory

Project Report

An Interactive 3D Model of the KUET Student Welfare Centre

Real-Time Rendering, Animation, Walk-Through and Ray Tracing
in C++ and OpenGL



Submitted by

Abdullah Al Mahmud
Roll: 2107099
Department of CSE, KUET

Course Teachers

Mr. Md. Mubtashim Abrar Nihal
Mr. Md. Tajmilur Rahman

October 2026

Abstract

This report describes an interactive three-dimensional model of the Student Welfare Centre (SWC) of Khulna University of Engineering & Technology. The model was reconstructed from 93 photographs and is written in C++17 with OpenGL 3.3. Every stage of the graphics pipeline is implemented in the project itself: the geometric primitives, the vector and matrix mathematics, the modelling, viewing and projection transformations, the Phong illumination model, shadow mapping, procedural surface materials, time-based animation, and first-person walking with collision detection. In addition to the real-time renderer, the same scene can be rendered by a CPU ray tracer that adds mirror reflections, exact shadows and ambient occlusion. The finished scene contains about 367,000 triangles, 113 point lights, 21 rotating fans, 7 moving vehicles and 35 walking pedestrians. The report explains each feature together with the equations behind it, shows the results, and states the limits of the work.

Contents

1 Introduction **2**

2 System Overview **2**

3 Geometric Modelling **3**

4 Viewing and Projection **6**

5 Illumination and Shading **7**

6 Animation **8**

7 Walking and Collision Detection **9**

8 Ray Tracing **9**

9 Results **11**

10 Limitations and Future Work **12**

11 Conclusion **13**

1 Introduction

1.1 Objective

The aim of the project is to build a virtual 3D environment using the techniques taught in the course, and to apply them to a real place. The Student Welfare Centre was chosen because it combines regular architecture (columns, window bays, corridors) with one distinctive form, a tall curved brick wall, and because it could be photographed inside and outside.

No 3D model files, image textures or mathematics libraries are used. The only external library is GLFW, which opens the window and reads the keyboard and mouse.

1.2 Requirements and how they are met

#	Requirement	Implementation
1	Virtual 3D environment	The SWC building, its interior, the site and the street behind it
2	Objects from basic primitives	Nine canonical shapes; all geometry is assembled from them (Sec. 3.1)
3	Translation, rotation, scaling	Custom 4×4 matrices (Sec. 3.3)
4	Composite transformations	TRS products, a transform stack, instanced moving objects (Sec. 3.4)
5	Custom vector and matrix operations	`Vec3`, `Mat4`, `Mat3`; dot, cross, product, inverse transpose
6	Camera and viewing	Custom look-at matrix; orbit and walking cameras; 37 preset views (Sec. 4)
7	Perspective projection	Custom perspective and orthographic matrices
8	Interactive manipulation	Mouse and keyboard camera; keys to move, rotate and scale the model
9	Ambient, diffuse, specular light	One sun and 113 point lights (Sec. 5)
10	Phong model, per-fragment	Evaluated in the fragment shader for every pixel
11	Exploring from viewpoints	Preset views, orthographic elevations, walking mode (Sec. 7)

Table 1: Course requirements and their implementation.

Features beyond the requirements are shadow mapping, procedural materials and sky, transparent glass, animation of fans, traffic and people, collision detection, and ray tracing.

2 System Overview

2.1 Structure of the program

The source is divided into modules with one responsibility each (Table 2). The scene is built once at start-up into a single array of vertices. A vertex holds eleven numbers: position, normal, colour, a gloss value and a pattern number that selects a procedural material. This array is uploaded to the graphics card once.

Module	Responsibility
`geometry/`	Vector and matrix mathematics; generators for the nine basic shapes; the vertex array and the transform stack
`scene/`	Materials and building dimensions; reusable parts (window bay, door, balcony panel); the building, interior, landscape, street, vehicles and people
`input/`	Orbit and walking cameras, preset views, keyboard and mouse handling
`navigation/`	Collision world for walking, and automatic route checks
`render/`	GLSL shaders, the OpenGL frame loop, the CPU ray tracer, image output
`tests/`	Unit tests for the shapes and for collision

Table 2: Modules of the program.

2.2 Rendering one frame

Each frame is produced in the following order.

1. Read input; advance the fan angle and the clock for traffic and people.
2. **Shadow pass:** draw the scene from the sun's position, storing depth only.
3. Draw the procedural sky.
4. Select the 24 point lights nearest the camera.
5. **Opaque pass:** draw the static scene, then each fan, vehicle and person with its own model matrix.
6. **Transparent pass:** draw the glass panes sorted from far to near, with blending.

The vertex array is laid out in ranges (static scene, one fan rotor, three vehicle meshes, ten person meshes, the glass panes), so a moving object is drawn by issuing one draw call for its range with a different matrix.

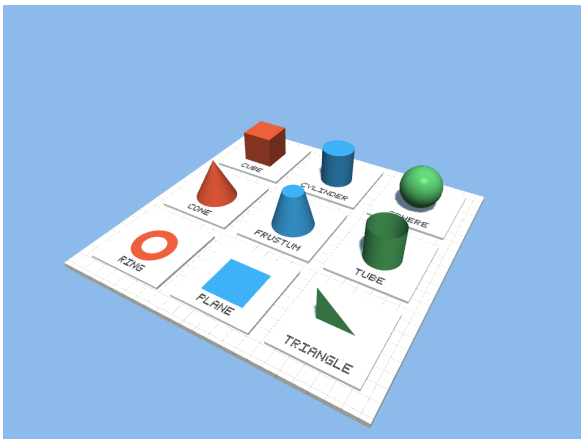


Figure 1: Aerial view of the complete scene: the building around its courtyard, the plaza, the campus roads, and the street with stalls behind.

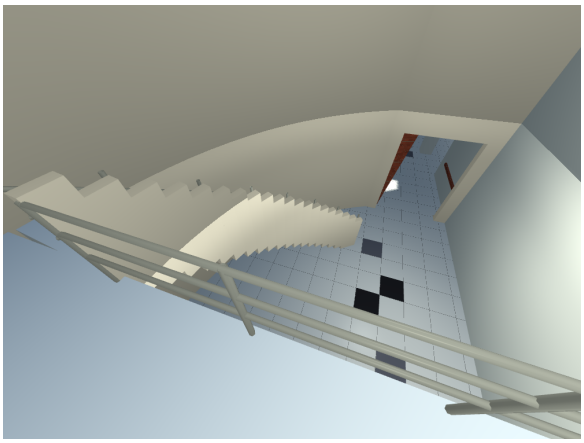
3 Geometric Modelling

3.1 Primitives and triangle meshes

Everything drawn is a triangle. A four-sided face (A, B, C, D) is split into the triangles (A, B, C) and (A, C, D). Nine unit shapes are defined once and then sized and placed: cube, cylinder, sphere, cone, frustum, open tube, ring, plane and triangle (Fig. 2a).



(a) The nine basic shapes.



(b) Curved stair built from ring sectors.

Figure 2: Primitives, and a structure assembled from them.

Cylinder. With N segments, $\theta_i = 2\pi i/N$, a point on the surface is

$$\mathbf{p}(\theta, y) = \big(c_x + r_x \cos\theta,\ y,\ c_z + r_z \sin\theta\big). \tag{1}$$

Unequal radii $r_x \neq r_z$ give an elliptical cylinder.

Cone and frustum. The radius varies linearly with height, $r(t) = (1-t)r_0 + t\, r_1$ for $t \in [0,1]$; a cone has $r_1 = 0$.

Sphere. With latitude ϕ and longitude θ,

$$\mathbf{p} = \mathbf{c} + r\,(\cos\phi\cos\theta,\ \sin\phi,\ \cos\phi\sin\theta). \tag{2}$$

Scaling the axes unequally gives an ellipsoid, used for foliage.

Rod. A cylinder between two arbitrary points A and B, used for railings, branches and limbs. Its axis is $\hat{\mathbf{a}} = (B-A)/\|B-A\|$; with any helper vector $\mathbf{h}$ not parallel to it, $\mathbf{u} = \text{unit}(\hat{\mathbf{a}} \times \mathbf{h})$ and $\mathbf{v} = \hat{\mathbf{a}} \times \mathbf{u}$ span the circular cross-section.

Curved wall. The curved brick wall is a partial cylinder of 60×14 flat quads. A point at angle θ, height y and radius R about the centre (c_x, c_z) is

$$\mathbf{p} = (c_x + R\sin\theta,\ y,\ c_z + R\cos\theta). \tag{3}$$

Each vertex is given the true radial normal, so the lighting is smooth although the surface is faceted.

Curved stair. The stair inside the curved wall follows it, so each step is a sector of a ring between radii $r_{\text{in}} = 5.56$ and $r_{\text{out}} = 7.02$ (Fig. 2b). For a tread depth $d_t = 0.28$ on the middle radius $r_{\text{mid}} = 6.29$ and a riser $h_r = 3.6/20 = 0.18$, step i spans

$$\Delta\theta = \frac{d_t}{r_{\text{mid}}} \approx 2.55^\circ, \qquad \theta \in [\theta_0 + i\,\Delta\theta,\ \theta_0 + (i+1)\Delta\theta], \qquad y_i = 0.45 + (i+1)\,h_r. \tag{4}$$

Twenty steps turn through about 51° and rise one storey.

3.2 Vectors and matrices

All mathematics is written in the project. For vectors $\mathbf{a}, \mathbf{b}$:

$$\mathbf{a}\cdot\mathbf{b} = a_x b_x + a_y b_y + a_z b_z, \qquad \mathbf{a}\times\mathbf{b} = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x). \tag{5}$$

The dot product measures how closely two directions agree and drives all lighting. Writing $\text{unit}(\mathbf{a}) = \mathbf{a}/\|\mathbf{a}\|$ for a vector scaled to length one, the cross product gives a perpendicular vector; the normal of a flat triangle is $\mathbf{n} = \text{unit}((B-A)\times(C-A))$. Matrices are 4×4 and multiply as $(AB)_{ij} = \sum_k A_{ik}B_{kj}$.

3.3 Transformations

A point is written in homogeneous form $(x, y, z, 1)^T$ so that translation becomes a matrix:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{6}$$

A rotation about the X axis, R_x, is defined in the same way and is used, for example, to turn a cylinder on its side to make a wheel.

Composite transformation. An object is placed with

$$M = T\,R_y\,S, \tag{7}$$

which scales first, then rotates, then translates. The order matters: translating before rotating would swing the object around the origin.

Normals. Under non-uniform scaling a normal transformed by M is no longer perpendicular to the surface. With A the upper-left 3×3 part of M, normals are transformed by the inverse transpose:

$$\mathbf{n}' = \text{unit}\Big((A^{-1})^T\mathbf{n}\Big). \tag{8}$$

3.4 Hierarchy and instancing

Transform stack. A small class `Place` multiplies the current matrix by a local one and restores it when its scope ends:

$$M_{\text{child}} = M_{\text{parent}}\, M_{\text{local}}. \tag{9}$$

A window bay is therefore written once in its own coordinates, placed repeatedly along a wing, and the wing is placed on the site. The cafeteria front is modelled facing $+Z$ and a single `Place` turns it to face the road.

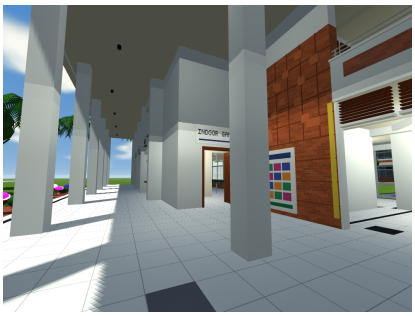
Instancing. One mesh is drawn many times with different matrices (Table 3). This is how 21 fans, 7 vehicles and 35 people are drawn from only 14 small meshes.

Mesh	Instances	Model matrix
Fan rotor	21	$M_{\text{scene}}\, T(\mathbf{h}_k)\, R_y(\theta(t))$
Easybike, cycle-van, bus	7	$M_{\text{scene}}\, T(x(t),\, 0.03,\, z_l)\, R_y(0 \text{ or } \pi)$
Person (5 colours $\times$ 2 poses)	35	$M_{\text{scene}}\, T(\mathbf{p}(t))\, R_y(\psi)$

Table 3: Instanced objects and their composite transformations.

3.5 The modelled spaces

The model follows the photographs space by space. Outside: the plaza with a circular *alpona* built from rings and petal shapes, the front colonnade and portico, the curved wall with pale bands, louvre strips and lettering that follows the curve, the courtyard with its stage, the cafeteria front with an external stair, and the shop wing. Inside: the entrance lobby with a terracotta relief wall, an indoor-games hall, the Shwapno shop, a cafeteria dining hall whose corridor wall is glass, tiled corridors with black accent tiles, two stairs, and a first-floor terrace under white portal frames. Behind the building: a boundary wall with two gates, a road with a central divider, and a row of stalls.



(a) Entrance lobby



(b) Cafeteria hall



(c) First-floor terrace



(d) Shwapno shop



(e) Courtyard and stage



(f) Campus gate

Figure 3: Some of the modelled spaces, rendered in real time.

4 Viewing and Projection

4.1 Orbit camera

The camera circles a target $\mathbf{c}$ at distance d with yaw ψ and pitch ϕ. Its eye position lies on a sphere:

$$\mathbf{e} = \mathbf{c} + d\,(\cos\phi\sin\psi,\ \sin\phi,\ \cos\phi\cos\psi). \tag{10}$$

Dragging the mouse changes ψ and ϕ; the wheel multiplies d by 0.9 per notch.

4.2 View matrix

The look-at matrix is built from the eye, the target and an up hint $\mathbf{u}_0$:

$$\mathbf{f} = \mathrm{unit}(\mathbf{c} - \mathbf{e}), \qquad \mathbf{s} = \mathrm{unit}(\mathbf{f} \times \mathbf{u}_0), \qquad \mathbf{u} = \mathbf{s} \times \mathbf{f}, \tag{11}$$

$$V = \begin{bmatrix} s_x & s_y & s_z & -\mathbf{s}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{12}$$

V moves the world so that the eye is at the origin looking along $-Z$.

4.3 Projection

For a vertical field of view α, aspect ratio a, near and far distances n, f, and $q = 1/\tan(\alpha/2)$, the perspective matrix is

$$P = \begin{bmatrix} q/a & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & -\dfrac{f+n}{f-n} & -\dfrac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}. \tag{13}$$

The last row copies depth into w; the division by w that follows makes distant objects smaller. The complete chain for a vertex is

$$\mathbf{p}_{\text{clip}} = P\,V\,M\,\mathbf{p}, \qquad \mathbf{p}_{\text{ndc}} = \mathbf{p}_{\text{clip},xyz} / p_{\text{clip},w}. \tag{14}$$

An orthographic matrix, which has no division by depth, is used for the architectural elevations and plan (Fig. 4a) and for the sun's shadow camera.



(a) Orthographic plan



(b) Perspective view of the street

Figure 4: The two projections. In (a) size is independent of distance; in (b) it is not.

5 Illumination and Shading

5.1 Phong illumination, per fragment

Lighting is computed for every pixel in the fragment shader, using the normal interpolated across the triangle. Let $\mathbf{n}$ be the normal, $\mathbf{l}$ the direction to the sun, $\mathbf{v}$ the direction to the eye, g the gloss of the material and $S_h \in [0,1]$ the shadow factor.

$$\text{Diffuse:} \quad D = \max(\mathbf{n}\cdot\mathbf{l}, 0)\, S_h, \tag{15}$$

$$\text{Specular:} \quad \mathbf{r} = 2(\mathbf{n}\cdot\mathbf{l})\mathbf{n} - \mathbf{l}, \qquad k = 8 + 88g, \qquad H = \max(\mathbf{v}\cdot\mathbf{r}, 0)^k\, g\, S_h, \tag{16}$$

$$\text{Ambient:} \quad \mathbf{A} = 0.62\ \mathrm{mix}(\mathbf{C}_{\text{ground}}, \mathbf{C}_{\text{sky}}, \tfrac{1}{2}n_y + \tfrac{1}{2}). \tag{17}$$

The final colour is

$$\mathbf{C} = \mathbf{C}_{\text{base}} \odot \left(\mathbf{A} + 0.95\,\mathbf{C}_{\text{sun}} D + \mathbf{L}_d\right) + 0.6\,\mathbf{C}_{\text{sun}} H + 0.5\, g\, \mathbf{L}_s, \tag{18}$$

where $\odot$ is component-wise multiplication and $\mathbf{L}_d, \mathbf{L}_s$ are the point-light sums defined below. The ambient term stands for light arriving from all directions: upward-facing surfaces take the colour of the sky and downward-facing ones the colour of the ground. A higher gloss gives a smaller, sharper highlight.

5.2 Point lights and emission

Sunlight does not reach the corridors and halls, so every ceiling fixture and stall lamp is a point light at $\mathbf{p}_i$ with colour $\mathbf{c}_i$ and radius r_i. For a surface point $\mathbf{p}$, with $d_i = \|\mathbf{p}_i - \mathbf{p}\|$ and $\mathbf{l}_i = (\mathbf{p}_i - \mathbf{p})/d_i$, the attenuation is

$$a_i = s_i\, \frac{\left(1 - (d_i/r_i)^4\right)^2}{1 + 0.22\, d_i + 0.20\, d_i^2}. \tag{19}$$

The denominator is the usual constant–linear–quadratic falloff. The numerator brings the light smoothly to zero at its radius, and s_i fades it out one storey below the fixture so that a lamp lights only its own floor.

$$\mathbf{L}_d = \sum_i \mathbf{c}_i\, a_i (0.25 + 0.75 \max(\mathbf{n}\cdot\mathbf{l}_i, 0)), \qquad \mathbf{L}_s = \sum_i \mathbf{c}_i\, a_i \max(\mathbf{v}\cdot\mathbf{r}_i, 0)^k. \tag{20}$$

The constant 0.25 stands in for light bounced from white walls. Lamps, fluorescent tubes, lit signs and window glazing seen from indoors are *emissive*: they output their own colour regardless of the lighting.

5.3 Shadow mapping

The scene is first drawn from the sun's position with an orthographic camera into a 4096×4096 depth texture. When a pixel is shaded, its world position is transformed into the sun's view; if it is farther from the sun than the stored depth, something lies in front of it and it is in shadow:

$$\mathbf{p}_L = P_L V_L\, (\mathbf{p} + 0.05\, \hat{\mathbf{n}}), \qquad \mathbf{q} = \tfrac{1}{2}\, \mathbf{p}_{L,xyz}/p_{L,w} + \tfrac{1}{2}. \tag{21}$$

A bias $\beta = 0.0006 + 0.0012\,(1 - \max(\mathbf{n}\cdot\mathbf{l}, 0))$ prevents a surface from shadowing itself. Nine neighbouring texels are tested and averaged (percentage-closer filtering), which softens the shadow edge:

$$S_h = \frac{1}{9} \sum_{i=-1}^{1} \sum_{j=-1}^{1} \Big[\, q_z - \beta \le D_{\text{map}}(q_x + \tfrac{i}{4096},\, q_y + \tfrac{j}{4096}) \Big]. \tag{22}$$

Fans, vehicles and people are drawn into the shadow map as well, so their shadows move with them.

5.4 Procedural materials

No image textures are used. The pattern number of a vertex selects a function of world position in the shader. For brick and paving, laid in running bond with units of size (w, h) at surface coordinates (u, v):

$$j = \lfloor v/h \rfloor, \qquad \mathbf{c} = (u/w + \tfrac{1}{2}(j \bmod 2),\ v/h). \tag{23}$$

The fractional part of $\mathbf{c}$ locates the pixel within its brick (near zero it is mortar); the integer part identifies the brick and seeds a hash that varies its colour. Floor tiles use the same idea on a straight 0.6 m grid. Grass, bark and thatch use value noise: random values on a grid blended with the weight $w(t) = t^2(3 - 2t)$. When a pattern becomes smaller than a pixel it is faded to its average colour, which removes shimmering. The sky blends from horizon to zenith and adds clouds from five octaves of noise; distant surfaces are fogged by $F = 1 - e^{-0.006\,\max(d-45,\,0)}$.

5.5 Transparent glass

Glass panes are drawn after all opaque geometry, sorted from far to near, with depth writing switched off:

$$\mathbf{C}_{\text{out}} = \alpha\,\mathbf{C}_{\text{glass}} + (1 - \alpha)\,\mathbf{C}_{\text{behind}}, \qquad \alpha = 0.20. \tag{24}$$

Glass also reflects more strongly at grazing angles, $F_g = 0.08 + 0.62\,(1 - \max(\mathbf{n} \cdot \mathbf{v}, 0))^3$.

6 Animation

All motion is driven by elapsed time, capped at 0.05 s per frame so that a pause does not cause a jump. Pressing `P` freezes all of it.

6.1 Ceiling fans

$$\theta \leftarrow (\theta + \omega\,\Delta t) \bmod 2\pi, \qquad \omega = 240^\circ/\text{s}. \tag{25}$$

The mount is static; only the rotor is drawn with $T(\mathbf{h}_k)\,R_y(\theta)$.

6.2 Road traffic

Four easybikes, two cycle-vans and one bus use the road behind the building (Fig. 5). A vehicle with speed v, offset s_0 and loop length L has travelled

$$s(t) = (s_0 + vt) \bmod L \tag{26}$$

round its loop. Only the first $D = 116$ m of the loop lies on the road, so the vehicle is drawn while $s \le D$, at $x = x_{\text{start}} + s$ in one lane and $x = x_{\text{end}} - s$ in the other. Traffic keeps to the left of the divider. Vehicles in a lane share v and L, so their spacing is constant and they never overlap. The bus is visible for $D/L \approx 32\%$ of the time, which makes it an occasional sight.



(a) Easybike **(b)** Cycle-van **(c)** Stalls across the road

Figure 5: The street behind the SWC, with moving vehicles.

6.3 Pedestrians

Thirty-five people walk fourteen routes inside and outside the building. A route is a polyline $\mathbf{w}_0, \dots, \mathbf{w}_n$ of total length L. The distance of a walker along it is

$$s(t) = \begin{cases} (s_0 + vt) \bmod L & \text{closed loop,} \\ L - \left| (s_0 + vt) \bmod 2L - L \right| & \text{open route (there and back).} \end{cases} \tag{27}$$

On the segment that contains s, at fraction u along it and with travel direction $\mathbf{d}$, the position and heading are

$$\mathbf{p} = (1-u)\,\mathbf{w}_i + u\,\mathbf{w}_{i+1}, \qquad \psi = \operatorname{atan2}(-d_z,\ d_x). \tag{28}$$

Because $\mathbf{p}$ is interpolated in all three coordinates, a route that climbs the entrance steps lifts the walker with it. Two meshes with opposite legs forward alternate about twice a second to suggest a stride.

7 Walking and Collision Detection

In walking mode the viewer moves through the model at eye height. Collision uses the same triangles that are drawn, so what is seen is what blocks.

Spatial grid. Triangles are stored in 2 m cells indexed by $(\lfloor x/2 \rfloor, \lfloor z/2 \rfloor)$, so each step tests only nearby ones.

Movement. The walker is a cylinder of radius 0.18 and height 1.78 with eyes at 1.6. From forward input f, sideways input r and yaw ψ,

$$\mathbf{d} = (r\cos\psi - f\sin\psi,\ 0,\ -r\sin\psi - f\cos\psi), \qquad \Delta\mathbf{p} = \hat{\mathbf{d}}\, v\, \Delta t, \tag{29}$$

with $v = 2.4$ m/s. Normalising $\mathbf{d}$ prevents diagonal movement from being faster. Each step is divided into pieces of at most 0.06 so that a thin wall cannot be skipped.

Finding the floor. For a nearly horizontal triangle ABC the barycentric coordinates of the walker in plan are found from

$$\begin{bmatrix} B_x - A_x & C_x - A_x \\ B_z - A_z & C_z - A_z \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} x - A_x \\ z - A_z \end{bmatrix}, \qquad y = A_y + u\,(B_y - A_y) + v\,(C_y - A_y). \tag{30}$$

The walker is over the triangle when $u \ge 0$, $v \ge 0$ and $u + v \le 1$. A floor up to 0.24 higher can be stepped onto, which is how stairs are climbed. If no floor is found the move is refused, so the walker cannot leave a balcony.

Obstacles. Other triangles are clipped to the height range of the body and projected to plan. The closest point on an edge AB to the walker's centre P is

$$t = \operatorname{clamp}\!\left(\frac{(P - A)\cdot(B - A)}{\|B - A\|^2}, 0, 1 \right), \qquad Q = A + t\,(B - A), \tag{31}$$

and the move is blocked if $\|P - Q\|$ is less than the body radius. When a move is blocked its X and Z parts are tried separately, so the walker slides along a wall instead of stopping.

8 Ray Tracing

A second renderer traces rays through the same triangles, materials, lights and camera, entirely on the CPU. It produces effects that the Phong rasteriser cannot (Fig. 6).



Figure 6: The cafeteria hall from one camera position. Left: real-time OpenGL. Right: ray traced. The ray tracer adds the reflection of windows and furniture in the polished floor, a shadow from every lamp, and darkening where surfaces meet.

8.1 Primary rays

Ray generation is projection run in reverse: the rasteriser maps points to pixels, the ray tracer maps pixels to directions. For a sample at (p_x, p_y) in a $W \times H$ image, with the camera axes $\mathbf{f}, \mathbf{s}, \mathbf{u}$ of Section 4,

$$x_n = \frac{2p_x}{W} - 1, \quad y_n = 1 - \frac{2p_y}{H}, \qquad \mathbf{d} = \operatorname{unit}\!\left(\mathbf{f} + x_n \, a \tan\tfrac{\alpha}{2}\, \mathbf{s} + y_n \tan\tfrac{\alpha}{2}\, \mathbf{u}\right). \tag{32}$$

Points on the ray are $\mathbf{o} + t\,\mathbf{d}$.

8.2 Ray–triangle intersection

The Möller–Trumbore method is used. A triangle with corner $\mathbf{p}_0$ and edges $\mathbf{e}_1, \mathbf{e}_2$ contains the points $\mathbf{p}_0 + u\,\mathbf{e}_1 + v\,\mathbf{e}_2$. Equating this with the ray and solving by Cramer's rule, with $\mathbf{s} = \mathbf{o} - \mathbf{p}_0$:

$$\mathbf{h} = \mathbf{d} \times \mathbf{e}_2, \quad \mathbf{q} = \mathbf{s} \times \mathbf{e}_1, \quad \Delta = \mathbf{e}_1 \cdot \mathbf{h}, \qquad u = \frac{\mathbf{s}\cdot\mathbf{h}}{\Delta}, \quad v = \frac{\mathbf{d}\cdot\mathbf{q}}{\Delta}, \quad t = \frac{\mathbf{e}_2\cdot\mathbf{q}}{\Delta}. \tag{33}$$

The ray hits when $u \ge 0$, $v \ge 0$, $u + v \le 1$ and $t > 0$. The same u, v interpolate the vertex normals.

8.3 Bounding-volume hierarchy

Testing 350,000 triangles for every ray is impractical, so the triangles are grouped into a binary tree of axis-aligned boxes. A ray that misses a box skips everything in it. The ray meets a box when its three slab intervals overlap:

$$t_{\text{in}} = \max_k \min\!\left(\frac{b_k^{\min} - o_k}{d_k}, \frac{b_k^{\max} - o_k}{d_k}\right), \quad t_{\text{out}} = \min_k \max\!\left(\frac{b_k^{\min} - o_k}{d_k}, \frac{b_k^{\max} - o_k}{d_k}\right), \quad t_{\text{out}} \ge \max(t_{\text{in}}, 0). \tag{34}$$

The tree is built with the surface-area heuristic: twelve candidate splits along the longest axis are compared and the one with the lowest cost $A_L N_L + A_R N_R$ is chosen, where A is the surface area of a child box and N its triangle count. The area of a box is proportional to the chance that a random ray hits it, so the cost estimates the work that remains.

8.4 Shadows, reflection and transmission

Shadow rays. From each hit point a ray is sent to the sun and to each nearby lamp; the light contributes only if the ray is unobstructed. The resulting shadows are exact, and every lamp casts them.

Reflection. For polished surfaces the ray is mirrored, $\mathbf{d}_r = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\mathbf{n}$, and traced again, up to three bounces. The fraction reflected follows Schlick's approximation of the Fresnel equations:

$$F = F_0 + (1 - F_0)(1 - \cos\theta)^5, \qquad \mathbf{C} = (1 - F)\,\mathbf{C}_{\text{local}} + F\,\mathbf{C}(\mathbf{d}_r). \tag{35}$$

A floor seen from above reflects a few per cent; seen at a grazing angle it is almost a mirror, as it is in the photographs.

Glass. A thin flat pane refracts a ray on entry and refracts it back on exit, so the ray leaves in its original direction. The tracer therefore continues the ray and keeps 20% of the pane's own colour, matching the blend of the rasteriser.

8.5 Ambient occlusion and anti-aliasing

Eight short rays are sent over the hemisphere above each visible point and the ambient term is reduced by the fraction that are blocked, $\mathbf{A}' = \mathbf{A}\,(1 - 0.70\,N_{\text{blocked}}/N)$. Directions are cosine-weighted: from random numbers ξ_1, ξ_2,

$$\boldsymbol{\omega} = \sqrt{\xi_1}\cos(2\pi\xi_2)\,\mathbf{t} + \sqrt{\xi_1}\sin(2\pi\xi_2)\,\mathbf{b} + \sqrt{1-\xi_1}\,\mathbf{n}. \tag{36}$$

Each pixel is divided into a $g \times g$ grid with one jittered sample per cell, and the results are averaged. Image rows are rendered in parallel by one thread per processor.

	Rasteriser	Ray tracer
Visibility	Depth buffer	Nearest hit in the hierarchy
Sun shadows	Shadow map, 3×3 filter	Shadow ray, exact
Lamp shadows	None	One shadow ray per lamp
Reflections	Estimated, glass only	Traced, all polished surfaces
Ambient light	Constant per direction	Reduced by occlusion rays
Anti-aliasing	$4\times$ multisampling	Jittered supersampling
Speed	Real time	Seconds to minutes per frame

Table 4: Comparison of the two renderers.

9 Results

9.1 Comparison with the photographs

Camera presets were placed where the photographs were taken, so that the model can be judged against them (Fig. 7).

9.2 Scene statistics and interaction

Quantity	Value	Quantity	Value
Reference photographs	93	Point lights	113
Triangles	about 367,000	Ceiling fans	21
Source lines (C++/GLSL)	about 6,200	Vehicles	7
Preset views	37	Pedestrians	35
Shadow map	4096×4096	Automatic tests	3

Table 5: Size of the finished project.

The user can orbit, pan and zoom with the mouse; rotate, scale and translate the whole model with the keyboard; jump to any of 37 views; walk through the building with `WASD`; toggle shadows; pause the animation; and ray trace the current view with a single key. Any view can also be saved to an image file from the command line, with either renderer.

9.3 Testing

Three automatic tests are provided. The first checks the triangle counts, bounds and normals of the basic shapes. The second checks collision in a small test world: doorways pass, walls block, movement slides along walls, stairs change the eye height, and low ceilings block. The third walks fixed routes through the full model, including both stairs up and down, every hall, the terrace and

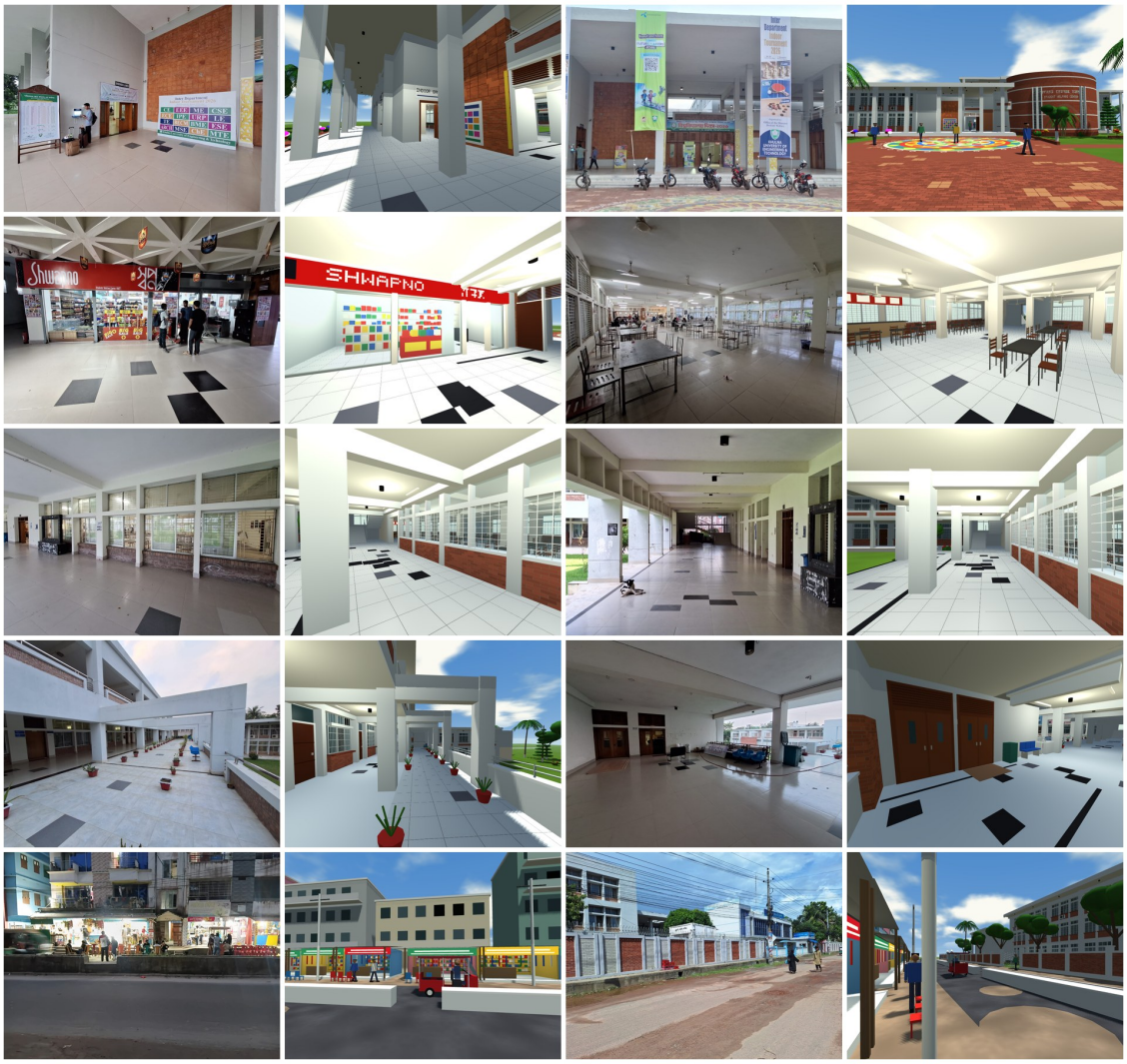


Figure 7: Photographs (left of each pair) beside the model (right): lobby and entrance; shop and cafeteria; corridor windows and corridor-end stair; terrace and first-floor hall; stalls and boundary wall.

the link between the wings, and confirms that glass walls and parapets stop the walker. All three pass. The visual match with the photographs, and the appearance of the moving objects, were judged by eye; no test compares pixels.

10 Limitations and Future Work

- **Dimensions are estimated** from photographs; nothing was measured on site. The second-floor interior and the rooms behind closed doors were not photographed and are not modelled.
- **The street is partly generated.** Its position relative to the building is assumed, and the stalls and town buildings are produced by rule with varied sizes and colours, not copied one by one.
- **Real-time lighting is simplified.** Point lights cast no shadows in the rasteriser, and only the 24 nearest are used. Window glazing seen from inside is a bright panel, so sunlight does not enter through it.
- **The ray tracer is not physically complete.** It has no soft shadows, no refraction through thick glass, and no indirect light beyond the ambient and fill terms.
- **Motion is simple.** Vehicles and people follow fixed paths, do not avoid each other, and are not part of collision. Wheels do not spin, and a walking figure alternates between two poses.

Natural extensions are shadowed point lights, a skeletal walk cycle, path tracing for indirect light, and a surveyed plan to replace the estimated dimensions.

11 Conclusion

The project meets all eleven requirements of the course with code written from first principles: primitives, vector and matrix mathematics, composite and hierarchical transformations, a look-at camera, perspective projection, interactive control and per-fragment Phong lighting. On that foundation it builds a recognisable model of a real campus building, inside and out, that can be explored on foot, and adds shadow mapping, procedural materials, transparency, animation and collision detection. Implementing a ray tracer over the same scene made the difference between the two rendering approaches concrete: the rasteriser is fast but approximates shadows and reflections, while the ray tracer computes them directly at a much higher cost.

References

[1] B. T. Phong, "Illumination for computer generated pictures," *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.

[2] L. Williams, "Casting curved shadows on curved surfaces," *Computer Graphics (SIGGRAPH '78)*, vol. 12, no. 3, pp. 270–274, 1978.

[3] T. Whitted, "An improved illumination model for shaded display," *Communications of the ACM*, vol. 23, no. 6, pp. 343–349, 1980.

[4] T. Möller and B. Trumbore, "Fast, minimum storage ray-triangle intersection," *Journal of Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997.

[5] C. Schlick, "An inexpensive BRDF model for physically-based rendering," *Computer Graphics Forum*, vol. 13, no. 3, pp. 233–246, 1994.

[6] D. Hearn, M. P. Baker and W. R. Carithers, *Computer Graphics with OpenGL*, 4th ed. Pearson, 2011.

[7] GLFW: an open-source library for OpenGL window and input handling. https://www.glfw.org